

CMT vs Vision Mamba

CNN-Transformer 하이브리드와 State Space Model의
심층 비교 분석 보고서

대상 논문

CMT: Convolutional Neural Networks Meet Vision Transformers (Guo et al., 2022)

Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model
(Zhu et al., 2024)

분석 범위

아키텍처 · 복잡도 유도 · Receptive Field · ZOH 이산화 · 수학적 접근 · 동작 시각화

목차

서론 — 두 모델의 문제 의식과 분기점

- 1.1 ViT의 한계에 대한 공통 인식
- 1.2 해결 방향의 분기 — Hybrid vs Replacement

아키텍처 구조 비교

- 2.1 전체 구조 — 계층적 vs 평면적
- 2.2 CMT Block의 3가지 구성요소
- 2.3 Vim Block의 작동 방식

복잡도 유도 과정 검증

- 3.1 표준 Self-Attention의 복잡도
- 3.2 CMT Block 복잡도 검증
- 3.3 Vim SSM 복잡도 검증
- 3.4 Cross-over Point 계산
- 3.5 CMT 실측 효율 vs Vim 이론적 효율

두 모델의 동작 메커니즘

- 4.1 CMT의 정보 흐름
- 4.2 Vim의 양방향 SSM 동작
- 4.3 정보 통합 방식의 본질적 차이

성능 차이 심층 분석

- 5.1 ImageNet 정확도 차이의 원인 분해
- 5.2 해상도별 효율 분석 (실측)
- 5.3 Detection/Segmentation 비교의 한계
- 5.4 ADE20K Segmentation
- 5.5 두 모델의 약점 시나리오

Receptive Field 비교 분석

- 6.1 CMT의 Receptive Field 구성
- 6.2 Vim의 Receptive Field 구성
- 6.3 Receptive Field의 질적 차이
- 6.4 Effective Receptive Field의 수학적 분석

Discretization Scheme (ZOH)의 영향

- 7.1 연속에서 이산으로
- 7.2 Zero-Order Hold 유도
- 7.3 ZOH의 정성적 영향

7.4 ZOH vs 다른 이산화 기법

7.5 Δ 학습의 실용적 영향

7.6 CMT의 암묵적 이산화

수학적 접근의 근본 차이

8.1 통일된 시각 — 토큰 간 정보 전파 함수

8.2 CMT의 수학적 토대

8.3 Vim의 수학적 토대

8.4 두 접근의 수학적 등가성

8.5 정보 이론적 관점

8.6 학습 동역학의 차이

Attention / State 시각화 분석

9.1 분류 Task에서의 주의 패턴

9.2 Detection Task — 큰 객체 vs 작은 객체

9.3 Semantic Segmentation의 픽셀 단위 동작

9.4 시각화 도구 및 비교 실험 설계

비교 논문 작성을 위한 실험 제안

10.1 권장 통제 실험

10.2 시각화 실험 제안

종합 평가와 후속 연구 맥락

1. 서론 — 두 모델의 문제 의식과 분기점

두 논문은 모두 "ViT의 한계를 어떻게 극복할 것인가"라는 동일한 문제 의식에서 출발하지만, 해결 방향이 정반대입니다. CMT는 "Transformer에 CNN을 더하자"는 하이브리드 접근이고, Vim은 "Attention 자체를 SSM으로 대체하자"는 패러다임 전환에 가깝습니다.

1.1 ViT의 한계에 대한 공통 인식

CMT(2021)는 ViT의 세 가지 한계를 명시적으로 지적합니다.

- 이미지를 16×16 패치로 자르면 패치 내부의 2D 구조와 지역 정보가 손실됨
- 고정 패치 크기 때문에 multi-scale 표현이 어려워 dense prediction에 부적합
- self-attention이 입력 해상도에 대해 $O(N^2C)$ 로 증가해 고해상도 처리가 비현실적

Vim(2024)은 동일한 quadratic complexity 문제를 인식하지만, "self-attention 자체가 필요하지 않다"고 주장합니다. Mamba가 언어 모델에서 보여준 linear scaling을 시각으로 가져오되, Mamba의 두 가지 약점인 (i) 단방향 모델링과 (ii) 위치 인식 부재를 극복하는 것이 목표입니다.

1.2 해결 방향의 분기 — Hybrid vs Replacement

두 논문의 근본적 가정 차이: CMT는 "attention은 유지하되 보완해야 한다", Vim은 "attention은 대체 가능하다".

2. 아키텍처 구조 비교

2.1 전체 구조 — 계층적 vs 평면적

항목	CMT-S	Vim-S
전체 구조	계층적(Hierarchical) 4-stage (ResNet/ EfficientNet 스타일)	Columnar(평면적) 단일 스케일 (ViT 스타일)
Stem	3×3 conv stride 2 + 3×3 conv ×2 (점진적 다운샘플)	16×16 단일 patch projection
블록 수	Stage별 [3, 3, 16, 3]	24개 블록 stack
다운샘플링	각 stage 사이 2×2 conv stride 2로 2배 해상도 감소	없음 (모든 블록 동일 해상도)
출력 feature 해상도	H/4, H/8, H/16, H/32 (multi-scale)	H/16 단일 (single-scale)
분류 토큰	Global Average Pooling	Middle class token (ablation으로 최적 입증)

CMT는 CNN의 전통적 design philosophy를 따라 해상도는 줄이고 채널은 늘리는 피라미드 구조를 갖습니다. 이는 detection/segmentation 같은 dense prediction에 FPN 등을 자연스럽게 붙일 수 있다는 실용적 장점이 있습니다. 반면 Vim은 ViT처럼 전체 시퀀스 길이를 유지하는 평면 구조입니다. 이는 ViTDet/MAE류 modality-agnostic 사전학습 패러다임과 정합성이 높다는 의미입니다.

2.2 CMT Block의 3가지 구성요소

(1) LPU (Local Perception Unit)

$$\text{LPU}(X) = \text{DWConv}(X) + X$$

3×3 depth-wise conv로 지역 정보를 주입. ViT의 absolute positional encoding이 translation-invariance를 깨는 문제를 자연스럽게 해결하는 동시에 패치 내부 구조 정보를 보존합니다.

(2) LMHSA (Lightweight Multi-Head Self-Attention)

K, V에 stride k의 depth-wise conv를 적용해 공간 크기를 $1/k^2$ 로 축소한 뒤 attention 수행:

$$\text{LightAttn}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k} + B) V'$$

학습 가능한 **relative position bias B**를 도입하여 bicubic interpolation을 통해 다른 해상도로 fine-tuning 가능. Stage별로 k=8, 4, 2, 1로 설정.

(3) IRFFN (Inverted Residual FFN)

$$\text{IRFFN}(X) = \text{Conv}(F(\text{Conv}(X))), \quad F(X) = \text{DWConv}(X) + X$$

MobileNetV2의 inverted residual block에서 영감. 1×1 conv로 확장 $\rightarrow 3 \times 3$ DWConv $\rightarrow 1 \times 1$ conv로 축소. Shortcut의 위치를 변경한 점이 ablation에서 0.6%p 성능 차이를 만듭니다.

2.3 Vim Block의 작동 방식

입력 시퀀스를 LayerNorm 후 x, z 로 분기. x 에 대해 forward/backward 양방향으로 각각 다음을 수행:

- 1D Conv $\rightarrow$ SiLU
- B, C, Δ 선형 투영
- discretization (zero-order hold): $\bar{A} = \exp(\Delta A)$, $\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$
- SSM 연산: $h_t = \bar{A} h_{t-1} + \bar{B} x_t$, $y_t = C h_t$

두 방향의 출력을 z 로 게이팅한 뒤 합산. **이 양방향성이 핵심 혁신**입니다. Ablation에서 단방향 $\rightarrow$ 양방향 SSM + Conv1d로 변경 시 ImageNet 73.2 $\rightarrow$ 73.9%, ADE20K 32.3 $\rightarrow$ 35.9 mIoU로 segmentation에서 특히 큰 향상.

블록 단위 차이: CMT는 LPU(지역) + LMHSA(전역) + IRFFN(채널 mix)의 3단계 모듈식 설계. Vim은 양방향 SSM이 지역·전역·시퀀스 모델링을 하나의 연산으로 통합.

3. 복잡도 유도 과정 검증

3.1 표준 Self-Attention의 복잡도 (기준선)

입력 $X \in \mathbb{R}^{(n \times d)}$ 에 대해, 표준 MHSA의 연산을 항별로 분해합니다.

Step 1: Q, K, V 투영 — 각 $W \in \mathbb{R}^{(d \times d)}$. Q, K, V 모두 합쳐 $3nd^2$ FLOPs.

Step 2: Attention score QK^T — $Q \in \mathbb{R}^{(n \times d)}$, $K^T \in \mathbb{R}^{(d \times n)}$ 곱으로 결과는 $n \times n$. FLOPs = n^2d .

Step 3: $\text{Softmax}(\cdot)V$ — $n \times n$ 과 $n \times d$ 의 곱. FLOPs = n^2d .

Step 4: 출력 투영 — nd^2 FLOPs.

합산:

$$O(\text{MHSA}) = 3nd^2 + n^2d + n^2d + nd^2 = 4nd^2 + 2n^2d$$

CMT 논문 식 (10)은 d_k, d_v 를 분리한 형태로 표현하지만 $d_k = d_v = d$ 대입 시 동일. Vim 논문 식 (7)의 $4MD^2 + 2M^2D$ 와 정확히 일치합니다.

FFN (식 11): $r=4$ 일 때 $O(\text{FFN}) = 8nd^2$

Transformer 블록 총합:

$$O(\text{Trans block}) = 12nd^2 + 2n^2d \quad \checkmark$$

3.2 CMT Block 복잡도 검증

LPU: 3×3 DWConv는 채널별 독립이므로 $O(\text{LPU}) = 9nd \quad \checkmark$

LMHSA: K, V에 $k \times k$ stride k DWConv 적용으로 공간 크기가 $n \rightarrow n/k^2$ 축소.

- Q 투영: nd^2
- K, V 투영 (축소 후): $2nd^2/k^2$
- QK'^T : $n \times (n/k^2) \times d = n^2d/k^2$
- $\text{Attention} \cdot V'$: n^2d/k^2
- 출력 투영: nd^2

$$O(\text{LMHSA}) \approx 2nd^2(1 + 1/k^2) + 2n^2d/k^2 \quad \checkmark \text{ (식 14와 일치)}$$

IRFFN: 1×1 conv (확장 $\times 4$) $\rightarrow 3 \times 3$ DWConv $\rightarrow 1 \times 1$ conv

$$O(\text{IRFFN}) = 8nd^2 + 36nd \quad \checkmark \text{ (식 15와 일치)}$$

CMT 블록 총합:

$$O(\text{CMT}) = 10nd^2(1 + 0.2/k^2) + 2n^2d/k^2 + 45nd$$

핵심 관찰: n^2 항이 $2n^2d/k^2$ 로 살아있습니다. 즉 CMT는 quadratic을 제거하지 못하고 계수를 $1/k^2$ 만큼 줄였을 뿐입니다. Stage 1에서 $k=8$ 이면 $1/64$ 로 축소되어 사실상 큰 영향이 없지만, Stage 4에서 $k=1$ 이면 표준 attention과 동일한 quadratic.

3.3 Vim SSM 복잡도 검증

Vim Algorithm 1을 따라, 시퀀스 길이 M , hidden dim D , 확장 후 $E = 2D$, SSM state N 기준으로 계산.

- 선형 투영 (x, z): $2MDE$
- Conv1d on x' : $K_c \cdot M \cdot E$ (상수항)
- B, C 투영: $2MEN$
- Δ 투영: ME
- Discretization $\bar{A}, \bar{B}$: $M \cdot E \cdot N$
- SSM recurrence ($h_t = \bar{A} h_{t-1} + \bar{B} x_t, y_t = C h_t$): $2MEN$

Vim 논문 식 (8) 한 방향 기준:

$$\Omega(\text{SSM}) = 3M(2D)N + M(2D)N = 8MDN$$

양방향이므로 실제로는 약 2배 (Vim 논문은 단방향 표시).

3.4 Cross-over Point 계산

언제 SSM이 self-attention보다 유리해질까요? 두 식을 같다고 놓고 풀어봅니다.

$$4MD^2 + 2M^2D = 8MDN \rightarrow M = 4N - 2D$$

$N=16, D=384$ (Vim-S 기준)이면 $M = 64 - 768 < 0$. 음수가 나오는 것은 모든 시퀀스 길이에서 SSM이 이미 유리하다는 의미가 아니라, 충분히 작은 D 에서만 SSM의 선형 항이 의미가 있고, 큰 D 에서는 quadratic 항이 늦게야 지배하기 시작한다는 것입니다.

더 정확히, **quadratic 항이 linear 항을 추월하는 지점:**

$$2M^2D = 4MD^2 \rightarrow M = 2D$$

D=384이면 M = 768. 즉 시퀀스 길이가 768을 넘어가면 self-attention의 quadratic 항이 본격적으로 지배하기 시작합니다.

입력 해상도	패치 16 기준 시퀀스 길이 M	Quadratic 지배?
224×224	196	아니오 (linear 항이 지배)
384×384	576	경계
512×512	1,024	예
1024×1024	4,096	확연히 예
1248×1248	6,084	압도적으로 예

이것이 Vim 논문 Figure 1에서 보는 현상의 정량적 근거입니다 — 512²에서는 Vim과 DeiT가 비슷하지만, 1248²에서는 2.8× 차이가 발생합니다.

3.5 CMT의 실측 효율 vs Vim의 이론적 효율

D=384, N=16 가정 하에 시퀀스 길이별 FLOPs (단위: G):

M	Self-Attention (ViT/DeiT)	CMT LMHSA (k=2)	Vim SSM (양방향)
196	~0.14	~0.11	~0.02
1,024	~1.41	~0.68	~0.10
2,048	~4.42	~1.57	~0.20
4,096	~15.21	~4.41	~0.40
6,084	~32.55	~9.31	~0.60

M=196 (224² 입력)에서는 세 곡선이 비슷한 영역에 있어 차이가 크지 않습니다. 그러나 M=2048부터 self-attention이 급격히 발산하기 시작하고, M=6084(Vim 논문의 1248² 입력)에서는 SSM이 self-attention보다 약 **50배 저렴**합니다. CMT의 LMHSA는 중간 영역에 위치하지만 k=2 기준 quadratic 항이 여전히 살아있어 장기적으로는 self-attention과 동일한 점근적 거동을 보입니다.

4. 두 모델의 동작 메커니즘

4.1 CMT의 정보 흐름

CMT 블록 한 번의 정보 흐름은 다음과 같습니다:

1. **Input X** ($H \times W \times d$) $\rightarrow$ LPU: DWConv 3×3 로 지역 정보 + position-like 정보 주입
2. **LayerNorm** $\rightarrow$ LMHSA: Q는 그대로, K/V는 $k \times k$ DWConv stride k 로 $1/k^2$ 축소 후 $\text{Softmax}(QK^T/\sqrt{d_k} + B)V'$ 수행. B는 학습 가능한 relative position bias.
3. **+ Residual**
4. **LayerNorm** $\rightarrow$ IRFFN: 1×1 Conv (확장 $\times 4$) $\rightarrow 3 \times 3$ DWConv (locality 추가) $\rightarrow 1 \times 1$ Conv (축소). 내부 shortcut: $F(X) = \text{DWConv}(X) + X$.
5. **+ Residual $\rightarrow$ Output Y** (같은 해상도)

CMT 블록의 형식적 표현:

$$\begin{aligned} Y_i &= \text{LPU}(X_{i-1}) \\ Z_i &= \text{LMHSA}(\text{LN}(Y_i)) + Y_i \\ X_i &= \text{IRFFN}(\text{LN}(Z_i)) + Z_i \end{aligned}$$

4.2 Vim의 양방향 SSM 동작

Vim 블록 한 번의 처리 흐름 (Algorithm 1):

1. **Token sequence T** (**B, M, D**): $M = 196$ patches + class token (middle position)
2. **LayerNorm** $\rightarrow$ 선형 투영: $x, z \in (B, M, E=2D)$
3. **Forward path**: Conv1d $\rightarrow$ SiLU $\rightarrow B, C, \Delta$ 투영 $\rightarrow$ SSM(left $\rightarrow$ right) 계산 $\rightarrow y_{\text{forward}}$ 산출
4. **Backward path**: 동일 연산을 시퀀스 역방향으로 $\rightarrow y_{\text{backward}}$ 산출
5. **Gate path**: z 입력에 SiLU 적용
6. **Element-wise gating + merge**: $y_f \odot \text{SiLU}(z) + y_b \odot \text{SiLU}(z)$
7. **Linear projection back to D + Residual** $\rightarrow T_l$

Forward SSM의 수식 표현:

$$\begin{aligned} h_t &= \bar{A} h_{t-1} + B x_t \\ y_t &= C h_t \end{aligned}$$

여기서 $\bar{A}$, B 는 ZOH 이산화를 거친 행렬입니다 (7장에서 상세).

4.3 정보 통합 방식의 본질적 차이

Self-Attention (CMT의 LMHSA)

"모든 쌍을 직접 비교"

- Pairwise: $O(M^2)$ interactions
- Query t_3 가 $t_1, t_2, t_3, t_4, \dots$ 에 직접 가중치 부여
- 모든 토큰에 대한 직접 access
- 정보 손실 없음, 단 quadratic 비용
- Permutation-equivariant

Bidirectional SSM (Vim)

"고정 크기 상태에 과거를 압축"

- Sequential: $O(M)$ recurrence
- Hidden state $h \in \mathbb{R}^N$ ($N=16$, 고정)에 누적
- Past compressed into h + backward pass for future
- Linear complexity, 단 정보 병목 존재
- Position-aware by construction (Δ, B, C 가 x 에 의존)

Attention은 정보 손실이 없지만 quadratic 비용을 지불합니다. SSM은 hidden state $N=16$ 에 모든 과거를 압축해야 하므로 **정보 병목**이 있습니다. Mamba의 핵심 혁신은 입력 의존적 Δ, B, C 를 통해 "어떤 정보를 기억하고 잊을지"를 선택적으로 결정하는 것이고, Vim은 이를 양방향으로 두 번 수행해 미래 정보도 포착합니다.

CMT는 이 trade-off를 '계층적 구조 + K/V 축소'로 우회합니다. Stage 1처럼 토큰이 많은 곳에서는 K/V 를 $1/64$ 로 줄여 quadratic 비용을 낮추고, Stage 4처럼 토큰이 적은 곳에서는 full attention을 허용합니다. 이는 "해상도 별로 다른 전략"을 쓰는 셈입니다.

5. 성능 차이 심층 분석

5.1 ImageNet 정확도 차이의 원인 분해

같은 ~25M 파라미터 규모에서 CMT-S는 83.5%, Vim-S는 80.5%로 **3.0%p 차이**가 납니다. 이 차이를 어떤 요소가 만드는지 추정해 보겠습니다.

추정 기여 요인	효과 (대략)	근거
계층적 vs 평면 구조	+1.0~1.5%p	DeiT-S(79.8) → DeiT-S-4Stage(81.4)에서 1.6%p 향상 (CMT ablation Table 3)
CNN stem	+0.5%p	DeiT-S-4Stage(81.4) → +Stem(81.9)
LPU (locality)	+0.8%p	Table 5: +LPU(82.7)
IRFFN (vs vanilla FFN)	+0.6%p	Table 5: +IRFFN(83.3)
Long sequence FT	-1.1%p (Vim의 보완책)	Vim-S(80.5) → Vim-S † (81.6)
양방향성	+0.7%p	Vim Table 4: None(73.2) → Bidirectional SSM+Conv1d(73.9)

이 분해는 "Vim이 약한 게 아니라, CMT가 너무 많은 inductive bias를 주입한 것"에 가깝습니다. CMT의 +Stem +LPU +IRFFN +stage-wise는 모두 CNN 스타일의 **2D 지역성과 multi-scale을 강제하는 장치들**입니다.

만약 Vim에 동등한 inductive bias를 주입한 가상의 "Hierarchical Vim"이 있다면 (실제로 VMamba, LocalMamba 같은 후속 연구가 이 방향), 차이는 1%p 미만으로 축소될 가능성이 큼니다.

5.2 해상도별 효율 분석 (실측)

Vim 논문 Figure 1의 실측 데이터를 표로 정리:

해상도	DeiT-Ti FPS	Vim-Ti FPS	DeiT 메모리	Vim 메모리	Vim의 우위
512×512	~2.54 (log)	~2.29 (log)	4.56 GB	5.03 GB	DeiT가 약간 빠름
640×640	~2.25	~2.07	4.22 GB	8.09 GB	DeiT가 메모리 우위
738×738	~2.05	~1.91	8.13 GB	11.14 GB	비슷
1024×1024	~1.57	~1.71	12.48 GB	8.13 GB	Vim 추월 시작
1248×1248	~1.26	~1.71	40.09 GB	5.03 GB	2.8× 빠름, -86.8% 메모리

해석:

- 작은 해상도($\leq 738^2$)에서는 DeiT가 약간 유리. 이는 양방향 SSM의 하드웨어 최적화 오버헤드(B, C, Δ 투영, scan 연산)가 quadratic 항이 작을 때는 상대적으로 큰 비중을 차지하기 때문.
- 큰 해상도($\geq 1024^2$)에서 Vim의 우위가 폭발적. Cross-over가 약 768~1024 토큰 시퀀스 길이에서 발생, 이는 앞서 이론 분석에서 도출한 $M \approx 2D = 768$ 과 정확히 일치.

이것은 두 논문이 가진 **응용 영역의 분기점**을 의미합니다:

- 일반 분류(224^2) → CMT 압도적 우위
- Detection/Segmentation(800^2 내외) → 두 모델 비슷, CMT가 정확도 우위
- Gigapixel/위성/병리 영상(1000^2+) → Vim만 실용적

5.3 Detection/Segmentation 비교의 정직한 한계

이 비교는 다음 이유로 직접적이지 않습니다:

항목	CMT-S	Vim-Ti
Detection framework	RetinaNet (1× schedule)	Cascade Mask R-CNN
Backbone params	44.3M	~7M (Vim-S 미공개)
입력 해상도	1280×800	1024×1024
2D prior 사용?	자연스럽게 (4-stage hierarchical)	No (ViTDet-style sequence only)
Box AP	44.3	45.7
Mask AP	40.7	39.2

Vim-Ti가 CMT-S보다 Box AP에서 1.4 높지만, Vim은 Cascade Mask R-CNN(더 강한 framework)을 사용하고 ViTDet의 무거운 사전학습 패러다임을 따릅니다. 반대로 mask AP는 CMT-S가 1.5 높습니다 — CMT의 multi-scale feature가 마스크 경계 처리에 유리한 것으로 추정됩니다.

5.4 ADE20K Segmentation

모델	Params	mIoU
DeiT-Ti + UperNet	11M	39.2
Vim-Ti + UperNet	13M	41.0 (+1.8)
DeiT-S + UperNet	43M	44.0
Vim-S + UperNet	46M	44.9 (+0.9)
ResNet-101 + UperNet	86M	44.9

흥미로운 관찰: **Vim-S가 ResNet-101과 동일한 mIoU를 절반 파라미터로 달성**. 그러나 CMT는 ADE20K를 보고 하지 않아 직접 비교 불가.

5.5 두 모델의 약점이 드러나는 시나리오

CMT가 불리한 시나리오

- 1024^2 이상 고해상도 처리 (quadratic blow-up)
- 멀티모달 사전학습 (CNN 구조가 텍스트/오디오 시퀀스와 부조화)
- 가변 길이/해상도 입력 (Stage별 hyperparameter 재조정 필요)
- 큰 모델로의 scaling — 논문은 CMT-L까지만 검증 (84.8%, 74.7M), 더 큰 모델의 scaling law는 불명

Vim이 불리한 시나리오

- 작은 데이터셋에서의 학습 (inductive bias 부족 → 과적합 가능성)
- 정밀한 multi-scale 출력이 필요한 dense prediction (FPN 통합 시 추가 처리 필요)
- 표준 분류 정확도가 최우선인 경우
- 실시간 추론에서 짧은 시퀀스 (224^2 등) — SSM 최적화 오버헤드가 상대적으로 큼

6. Receptive Field 비교 분석

Receptive field(수용 영역)는 "한 출력 뉴런이 입력의 어느 픽셀들에 의존하는가"를 의미합니다. 두 모델은 이를 매우 다른 방식으로 확보합니다.

6.1 CMT의 Receptive Field 구성

CMT는 receptive field를 세 가지 메커니즘의 중첩으로 확보합니다.

(a) Convolution 기반 점진적 확장

각 conv layer는 커널 크기 k , stride s 에 대해 receptive field가 다음 점화식을 따릅니다:

$$r_l = r_{l-1} + (k_l - 1) \cdot \prod_{i=1}^{l-1} s_i$$

CMT-S의 conv 구조를 따라가면:

단계	연산	커널	Stride	누적 stride	누적 RF (pixels)
Stem-1	3×3 conv $s=2$	3	2	2	3
Stem-2	3×3 conv $s=1$	3	1	2	7
Stem-3	3×3 conv $s=1$	3	1	2	11
Patch Embed (S1)	2×2 conv $s=2$	2	2	4	13
LPU (S1)	3×3 DWConv	3	1	4	21
Patch Embed (S2)	2×2 conv $s=2$	2	2	8	25
LPU (S2)	3×3 DWConv	3	1	8	41
Patch Embed (S3)	2×2 conv $s=2$	2	2	16	49
LPU (S3)	3×3 DWConv	3	1	16	81
Patch Embed (S4)	2×2 conv $s=2$	2	2	32	97
LPU (S4)	3×3 DWConv	3	1	32	161

즉 **convolution 경로만 따져도 Stage 4 끝에서 ~161 픽셀의 receptive field**를 갖습니다. 224^2 입력의 약 72%에 해당.

(b) LMHSA의 전역 attention

LMHSA는 비록 K/V 를 $1/k^2$ 로 축소하지만, attention 자체는 모든 쿼리가 모든 (축소된) key에 접근합니다. 따라서 LMHSA 한 번 통과만으로 Effective Receptive Field는 이론상 무한대(전역)가 됩니다.

다만 실제로는 attention weight의 분포가 sparse하기 때문에 effective receptive field는 제한적입니다. Stage 1에서 $k=8$ 이면 K/V가 1/64로 축소되어 — 즉 각 쿼리가 ~49개 K/V 토큰만 보게 됩니다 (Stage 1의 $56 \times 56 = 3136 \rightarrow 49$ 토큰).

(c) IRFFN의 추가 locality

IRFFN 내부의 3×3 DWConv도 매 블록마다 receptive field를 ~2 pixel씩 확장합니다. Stage 3에 16개 블록이 쌓이면 IRFFN만으로 누적 ~32 pixel 추가.

CMT의 receptive field 특성 정리:

- **이론적:** LMHSA로 인해 한 블록 만에 전역
- **실효적(ERF):** convolution이 강제하는 "가까운 픽셀에 더 집중" 편향 때문에 종 모양(Gaussian-like) 분포
- **이방성:** DWConv 때문에 2D 격자 구조에 정합

6.2 Vim의 Receptive Field 구성

Vim은 매우 다릅니다 — **양방향 SSM 한 번이면 전체 시퀀스가 receptive field**입니다.

Forward SSM의 recurrence를 풀면:

$$y_t = C \sum_{k=0}^{t-1} \bar{A}^k B x_{t-k}$$

즉 시점 t 의 출력은 과거 모든 입력 $x_0, x_1, \dots, x_{t-1}$ 의 가중합입니다. 단방향 SSM만으로 과거 전체가 receptive field에 들어옵니다.

양방향성의 효과: Backward SSM이 미래 전체도 포함시키므로, 한 블록 통과 후 시퀀스의 모든 토큰이 receptive field에 들어옵니다.

핵심 차이 — Effective vs Theoretical RF: SSM의 effective RF는 $\bar{A}^k$ 의 감쇠율에 의해 결정됩니다. $\bar{A}$ 의 eigenvalue λ_i 에 대해:

$$\bar{A}^k = \text{diag}(\lambda_1^k, \lambda_2^k, \dots, \lambda_N^k)$$

$|\lambda_i| < 1$ 이면 지수적으로 감쇠하므로, 멀리 떨어진 토큰의 영향은 작아집니다. Mamba의 selective mechanism은 Δ_t 를 입력 의존적으로 조절하여 이 감쇠율을 토큰별로 다르게 제어합니다 — 중요한 토큰은 크게 기억하고 무관한 토큰은 빠르게 잊습니다.

6.3 Receptive Field의 질적 차이

CMT의 ERF — Gaussian-like

- Theoretical RF: global
- Effective RF: concentrated (가운데로 집중)

Vim의 ERF — Directional decay

- Theoretical RF: full sequence
- Effective RF: exponential decay

- Anisotropy: 2D-aware (등방적, 상하좌우 동일)
- DWConv-induced locality bias

- Anisotropy: 1D scan-aligned (스캔 방향에 정합)
- Bidirectional SSM with Δ -selective

중요한 비대칭성:

- CMT의 ERF는 **2D 등방적(isotropic)** — 상하좌우 동일하게 감쇠
- Vim의 ERF는 **1D scan 방향에 정합** — row-major flatten 시 같은 행의 토큰은 가깝지만, 바로 위 행의 토큰은 시퀀스 상 $W(=14\sim56)$ 만큼 떨어져짐 → 지수 감쇠로 멀어짐

이 점이 **Vim의 잠재적 약점**입니다. 이미지에서 수직으로 인접한 픽셀이 시퀀스에서는 멀어지므로, 2D 공간 구조가 왜곡됩니다. VMamba가 등장한 이유가 바로 이것 — 4방향 scan으로 이 비대칭성을 완화하려는 시도.

6.4 Effective Receptive Field의 수학적 분석

Luo et al. (NeurIPS 2016)의 ERF 분석 framework를 두 모델에 적용해보면:

CMT의 ERF 추정: L 개의 conv 레이어를 stack하면 ERF는 $\sqrt{L}$ 에 비례. CMT-S의 conv 경로는 약 30~40개의 DWConv/conv layer를 거치므로 $ERF \propto \sqrt{40} \approx 6.3$ pixel의 표준편차를 갖는 Gaussian. 그러나 LMHSA가 매 블록에서 flat한 전역 컴포넌트를 더하므로, 최종 ERF는 "Gaussian + flat tail" 혼합.

Vim의 ERF 추정: SSM의 영향력은 $\bar{A}^k$ 의 감쇠로 결정. $\bar{A} = \exp(\Delta A)$ 에서 A 의 eigenvalue가 음수(안정성 보장)이므로, $\bar{A}$ 의 eigenvalue는 $(0, 1)$ 범위. 평균 $|\lambda|^k = e^{(-k/\tau)}$ 로 보면 characteristic length $\tau = -1/\ln|\lambda|$.

선택적 메커니즘은 Δ_t 를 통해 이 τ 를 **토큰별로 동적 조절**합니다:

- Δ_t 크면 → $\bar{A}_t = \exp(\Delta_t A)$ 가 더 작아짐 → 빠른 망각 → 짧은 ERF
- Δ_t 작으면 → 느린 감쇠 → 긴 ERF

이는 "중요한 객체에 대해서는 ERF를 확장"하는 입력 의존적 조절을 의미합니다.

7. Discretization Scheme (ZOH)의 영향

이 부분은 Vim 논문에만 명시적으로 나오지만, **SSM의 동작 원리를 이해하는 핵심**이므로 깊게 다룹니다.

7.1 연속 시스템에서 이산 시스템으로

원래 SSM은 연속 시간 선형 시스템:

$$\dot{h}(t) = A h(t) + B x(t), \quad y(t) = C h(t)$$

이를 이산 시점 $t = 0, \Delta, 2\Delta, \dots$ 에서 sampling하려면 **이산화(discretization)**가 필요합니다.

7.2 Zero-Order Hold (ZOH) 유도

ZOH 가정: 샘플링 간격 Δ 동안 입력 $x(t)$ 가 일정하다고 가정.

연속 시스템의 해는 다음과 같이 표현됩니다:

$$h(t + \Delta) = e^{A\Delta} h(t) + \int_0^\Delta e^{A(\Delta-\tau)} B \cdot x(t+\tau) d\tau$$

ZOH 가정 $x(t+\tau) = x(t)$ for $\tau \in [0, \Delta]$ 적용:

$$h(t+\Delta) = e^{A\Delta} h(t) + \left(\int_0^\Delta e^{A(\Delta-\tau)} d\tau \right) B \cdot x(t)$$

적분 계산: $u = \Delta - \tau$ 치환 시 $du = -d\tau$, 범위 $\tau: 0 \rightarrow \Delta \Rightarrow u: \Delta \rightarrow 0$

$$\int_0^\Delta e^{A(\Delta-\tau)} d\tau = \int_\Delta^0 e^{Au} (-du) = \int_0^\Delta e^{Au} du = A^{-1}(e^{A\Delta} - I)$$

따라서:

$$\bar{A} = e^{A\Delta}, \quad \bar{B} = A^{-1}(e^{A\Delta} - I) B = (\Delta A)^{-1}(e^{\Delta A} - I) \cdot \Delta B$$

Vim 논문 식 (2)와 정확히 일치합니다.

7.3 ZOH의 정성적 영향

(1) Stability의 자동 보장

A 의 eigenvalue가 음의 실수부를 가지면 (Mamba의 HiPPO 초기화로 보장), $e^{A\Delta}$ 의 eigenvalue 크기는 $|e^{\lambda\Delta}| = e^{\text{Re}(\lambda)\Delta} < 1$. 즉 **이산 시스템도 자동으로 stable**.

(2) Δ 의 시간 스케일 해석

Δ 는 "한 토큰이 얼마나 긴 시간 구간을 차지하는가"로 해석됩니다:

- $\Delta \rightarrow 0$ (작은 step): $\bar{A} = e^{A\Delta} \approx I + A\Delta$, $B \approx B\Delta$. 거의 identity recurrence — 상태 변화가 미미, 입력을 거의 무시.
- $\Delta \rightarrow \infty$ (큰 step): $\bar{A} \rightarrow 0$, $B \rightarrow A^{-1}$. 과거를 완전히 잊고 현재 입력에 완전 의존.

이는 **gating의 연속적 일반화**입니다. 실제 Mamba 논문은 LSTM/GRU의 gate가 ZOH 이산화의 특수한 형태임을 지적합니다.

(3) Selective Mechanism과의 결합

Mamba/Vim에서 Δ_t 는 입력 의존적:

$$\Delta_t = \text{softplus}(\text{Linear}_{\Delta}(x_t') + \text{Parameter}_{\Delta})$$

따라서 매 토큰마다 다른 이산화 step을 적용:

- 정보가 풍부한 토큰 $\rightarrow \Delta_t$ 작게 $\rightarrow$ 천천히 통합
- 무관한 토큰 $\rightarrow \Delta_t$ 크게 $\rightarrow$ 빠르게 망각

이것이 Mamba의 selective 특성의 본질입니다.

7.4 ZOH vs 다른 이산화 기법

기법	$\bar{A}$ 정의	특성
Forward Euler	$I + \Delta A$	1차 정확, $\ I + \Delta A\ > 1$ 이면 unstable
Backward Euler	$(I - \Delta A)^{-1}$	항상 stable, but 1차 정확
Bilinear (Tustin)	$(I - \Delta A/2)^{-1}(I + \Delta A/2)$	2차 정확, stable 보존, S4에서 사용
ZOH (Mamba/Vim)	$\exp(\Delta A)$	정확한 해 (continuous에서 직접 유도), stable 보존

Vim이 ZOH를 선택한 것은 다음 때문으로 추정됩니다:

1. **정확성**: ZOH는 연속 시스템의 정확한 이산화 (다른 기법은 근사)
2. **효율성**: A 가 대각행렬이면 $\exp(A\Delta) = \text{diag}(e^{a_i \cdot \Delta})$ 로 element-wise 계산 가능 $\rightarrow$ SIMD 친화
3. **Stability 자동 보장**

7.5 Δ 학습의 실용적 영향

Vim Algorithm 1의 line 11:

$$\Delta_o \leftarrow \log(1 + \exp(\text{Linear}_\Delta(x'_o) + \text{Parameter}_\Delta))$$

softplus를 통해 $\Delta_t > 0$ 보장. 초기값은 Mamba의 $dt_{\min}$, $dt_{\max}$ (보통 $[0.001, 0.1]$) 사이에서 균일 sampling.

파라미터 카운트 영향: Δ 는 입력 차원 $E=2D$ 에서 스칼라로 투영되므로 추가 비용 2D. 다른 SSM 파라미터(A, B, C) 대비 매우 작음.

왜 비전에서 Δ 가 중요한가: 이미지 시퀀스에서 "빈 배경 토큰" vs "객체 경계 토큰"은 매우 다른 정보 밀도를 갖습니다. Δ 학습을 통해 모델은 다음을 학습할 수 있습니다:

- 배경 토큰 $\rightarrow$ 큰 $\Delta \rightarrow$ 빠르게 다음 토큰으로 정보 흘려보냄
- 객체 토큰 $\rightarrow$ 작은 $\Delta \rightarrow$ 상태에 오래 머물

이것은 attention의 "중요 토큰에 높은 가중치"와 기능적으로 유사하지만, **공간 복잡도가 다릅니다** — attention은 모든 쌍에 가중치를 부여하지만 ($O(M^2)$), SSM은 매 토큰의 시간 스케일을 조절($O(M)$)할 뿐입니다.

7.6 CMT는 이산화 문제가 없는가?

CMT는 명시적 이산화가 없지만, **암묵적으로 동등한 처리**가 존재합니다:

- **Attention의 softmax:** $\text{Softmax}(QK^T/\sqrt{d_k})$ 는 "각 query에 대해 어떤 key를 얼마나 봐야 하는가"의 확률 분포. 이는 Δ_t 의 attention 버전.
- **DWConv의 stride:** Stage 사이의 stride-2 conv는 "공간을 어떻게 sub-sample하는가"의 hard-coded 결정. SSM의 Δ 가 learnable인 반면, CMT의 stride는 fixed.
- **LMHSA의 k (reduction ratio):** stage 1은 $k=8$, stage 4는 $k=1$. 이는 "어느 해상도에서 얼마나 압축할지"를 hand-tuned로 결정.

핵심 차이: CMT는 시간/공간 스케일을 인간이 설계, Vim은 데이터로부터 학습. 이것이 Vim의 "selective" 패러다임이 갖는 일반성의 근원이지만, 동시에 학습 난이도와 데이터 요구량이 증가하는 원인이기도 합니다.

8. 수학적 접근의 근본 차이

8.1 통일된 시각 — 토큰 간 정보 전파 함수

모든 sequence model을 $y_t = \sum_s w(t, s, X) \cdot V(x_s)$ 형태로 표현할 수 있습니다. 여기서 $w(t, s, X)$ 는 "토큰 s 의 정보가 토큰 t 에 얼마나 전달되는가"의 가중치.

모델	$w(t, s, X)$	특성
RNN	$\prod_{k=s+1}^t W$ (고정)	입력 무관, 곱셈적 감쇠
Convolution	$K_{\{t-s\}}$ (지원 영역 내)	입력 무관, 지역적
Self-Attention	$\exp(q_t \cdot k_s) / \sum \exp(q_t \cdot k_{s'})$	입력 의존적, 전역, content-based
CMT LMHSA	Self-Attention with K/V subsampled by k	입력 의존적, 전역, but K/V만 reduced
S4 (Vim 이전)	$C \tilde{A}^{\{t-s\}} B$ (입력 무관)	입력 무관, 전역, 위치 기반
Mamba/Vim SSM	$C_t \cdot \prod_{k=s+1}^t \tilde{A}_k \cdot B_s$	입력 의존적, 전역, 위치+content 결합

8.2 CMT의 수학적 토대

CMT 블록은 본질적으로 세 가지 다른 수학 객체의 합성입니다.

(1) LPU: 평행이동 동변(translation-equivariant) 선형 연산

$$\text{LPU}(X)[i, j] = X[i, j] + \sum_{\{(p, q) \in N_{\{3 \times 3\}}\}} W[p, q] \odot X[i+p, j+q]$$

DWConv는 **채널별 독립** 가정 하의 spatial mixing. 표현력은 제한적이지만 (d 개의 3×3 필터만 사용) 계산 효율이 매우 높음.

(2) LMHSA: Content-based query-key matching

쿼리 $Q \in \mathbb{R}^{(n \times d_k)}$, 축소된 key/value $K', V' \in \mathbb{R}^{(n/k^2 \times d_k)}$:

$$\text{LMHSA}(X) = \text{Softmax}(QK'^T / \sqrt{d_k} + B) V'$$

여기서 $B \in \mathbb{R}^{(n \times n/k^2)}$ 는 **학습 가능한 relative position bias**. 이는 absolute position embedding 없이 위치 정보를 attention에 주입하는 방법.

수학적 통찰: CMT는 attention을 두 종류의 정보로 분해합니다:

- Content matching: QK'^T (입력의 semantic 유사도)
- Position bias: B (입력 무관, 학습된 위치 prior)

이는 Swin Transformer의 relative position bias와 동일한 아이디어. 위치 정보가 attention과 additive로 결합됨.

(3) IRFFN: Channel mixing + locality residual

$$\text{IRFFN}(X) = W_2 \cdot \sigma(\text{DWConv}(W_1 X) + W_1 X)$$

여기서 W_1 은 1×1 conv (expand), W_2 도 1×1 conv (contract), DWConv는 3×3 . 수학적으로 이는 **MobileNetV2의 inverted residual** 구조와 동일. Manifold hypothesis 관점에서, 고차원으로 비선형 확장 후 저차원으로 투영하는 것이 표현력 보존에 유리하다는 가정.

8.3 Vim의 수학적 토대

Vim의 SSM은 **선형 시불변 동역학 시스템의 확장**입니다.

(1) 연속 시스템의 매개변수화

상태 $h \in \mathbb{R}^N$, 입력 $x \in \mathbb{R}$, 출력 $y \in \mathbb{R}$:

$$h'(t) = A h(t) + B x(t)$$

Mamba/Vim에서 A는 **대각 행렬로 제약**: $A = \text{diag}(a_1, \dots, a_N)$. 이는 다음을 의미:

- 각 상태 차원이 독립적으로 진화
- $\bar{A} = \exp(\Delta A)$ 계산이 element-wise $\rightarrow$ 매우 효율적
- 표현력은 감소하지만, N을 키워 보완 (Vim에서 $N=16$)

HiPPO theory에 따르면, A가 특정 형태일 때 SSM은 **다항식 기저로 과거 신호를 압축**합니다. 즉 hidden state h_t 는 과거 시퀀스의 Legendre/Chebyshev 다항식 계수와 같은 의미.

(2) Selective Mechanism의 수학

Vanilla S4는 A, B, C, Δ 가 모두 입력 무관 — 즉 **선형 시불변(LTI) 시스템**. Mamba/Vim의 혁신은 이들을 **입력의 존적으로** 만든 것:

$$B_t = \text{Linear}_B(x_t'), \quad C_t = \text{Linear}_C(x_t'), \quad \Delta_t = \text{softplus}(\text{Linear}_\Delta(x_t') + \theta_\Delta)$$

이로 인해 시스템은 **선형 시변(LTV)**:

$$h_t = \bar{A}_t h_{t-1} + B_t x_t$$

여기서 $\bar{A}_t, B_t$ 모두 시간에 따라 다름. 이는 RNN의 일반화이지만, **세 가지 핵심 차이**:

1. 각 상태 차원이 독립 $\rightarrow$ 병렬 scan 가능 ($O(\log M)$ depth)
2. ZOH 이산화 $\rightarrow$ stability 자동 보장

3. A가 학습되지만 low-rank/대각 제약 → 효율성

(3) 양방향성의 수학적 의미

Vim의 양방향 SSM:

$$\begin{aligned} y_t^{\text{forward}} &= \sum_{s \leq t} C_t \cdot \bar{A}_{\{t:s+1\}} \cdot B_s \cdot x_s \\ y_t^{\text{backward}} &= \sum_{s \geq t} C_t \cdot \bar{A}_{\{s:t+1\}} \cdot B_s \cdot x_s \end{aligned}$$

여기서 $\bar{A}_{\{t:s+1\}} = \bar{A}_t \cdot \bar{A}_{\{t-1\}} \cdot \dots \cdot \bar{A}_{\{s+1\}}$ (cumulative product).

최종 출력은 gated combination:

$$y_t = \text{SiLU}(z_t) \odot (y_t^{\text{forward}} + y_t^{\text{backward}})$$

이는 attention의 bidirectional self-attention과 정보 흐름 관점에서 유사하지만:

- Attention: 모든 쌍 직접 비교
- Bidirectional SSM: 시퀀스 양 방향 누적 + gating

8.4 두 접근의 수학적 등가성 분석

언제 SSM이 attention과 동등한 표현력을 갖는가?

Sun et al. (2023, RetNet)과 후속 연구들이 보인 결과: **SSM은 attention의 특수한 경우**입니다.

Linear attention의 일반 형식:

$$y_t = \sum_{s \leq t} \varphi(q_t)^T \varphi(k_s) v_s$$

여기서 ϕ 는 feature map. SSM은 다음 선택으로 환원 가능:

$$\varphi(q_t)^T \varphi(k_s) = C_t \cdot \bar{A}_{\{t:s+1\}} \cdot B_s$$

즉 SSM은 **kernel을 $\bar{A}$ 의 cumulative product로 인수분해한** linear attention. 이는 다음을 의미:

- 표현력: $\text{SSM} \subset \text{linear attention} \subset \text{softmax attention}$
- 효율성: $\text{SSM} > \text{linear attention} > \text{softmax attention}$

CMT의 위치: CMT의 LMHSA는 softmax attention (단지 K/V만 subsampling). 따라서 표현력이 SSM보다 strict하게 더 큼 — 적어도 이론상으로는.

그렇다면 왜 동급 파라미터에서 CMT가 우위인가? 다음 요인이 결합된 결과로 추정:

1. CMT의 inductive bias (2D locality, multi-scale)가 ImageNet 규모에서 매우 유리
2. Vim은 일반성을 위해 inductive bias를 의도적으로 줄임 → 데이터 효율은 손해, 일반화는 이득

3. SSM의 정보 병목 ($N=16$) — 매우 긴 시퀀스에서는 정보 손실 발생

8.5 정보 이론적 관점

Channel capacity 비교:

- Self-Attention의 정보 채널: $O(M^2 \cdot d_k)$ 차원의 score matrix → 매우 큰 정보 용량
- SSM의 정보 채널: hidden state $h \in \mathbb{R}^N$, $N=16 \rightarrow$ 토큰 간 정보 전달 채널의 차원이 16

직관적으로 "긴 시퀀스에 $N=16$ 의 좁은 파이프를 통해 정보를 흘려보내는" 구조. Mamba의 selective mechanism은 이 좁은 파이프를 효율적으로 사용하기 위한 압축 메커니즘.

이는 Vim의 근본적 한계이기도 합니다. 시퀀스가 매우 길어지고 정보가 매우 dense하면, $N=16$ 상태로는 부족할 수 있습니다. 후속 연구들이 N 을 늘리거나 multi-head SSM을 도입하는 이유.

8.6 학습 동역학(Optimization)의 차이

CMT의 학습 동역학

- Loss landscape: convolution의 spatial locality + attention의 전역성이 결합되어 보통의 ResNet 학습과 유사한 동역학
- Gradient flow: 매 stage마다 downsampling이 있어 gradient가 자연스럽게 변환됨
- 학습 안정성: BN(in IRFFN, projections) + LN(before LMHSA, IRFFN) 혼합으로 안정적
- Mode collapse 위험: 낮음 (attention의 softmax가 자동 정규화)

Vim의 학습 동역학

- Loss landscape: SSM의 recurrence가 매우 깊은 effective depth (시퀀스 길이만큼)을 만들어 vanishing/exploding gradient 위험
- $\bar{A}$ 의 안정성이 핵심: 학습 초기 A 의 초기화가 부적절하면 학습 실패
- HiPPO 초기화 ($A = \text{Legendre}$ 또는 LegT 행렬)가 거의 필수
- BatchNorm 없이 LN만 사용 (Mamba 전통)

실험적 함의: Vim은 학습 hyperparameter에 더 민감합니다. Vim 논문이 300 epochs (DeiT 표준) 학습 후 추가로 long sequence fine-tuning 30 epochs를 한 이유는 SSM 학습의 점진성 때문으로 추정됩니다.

9. Attention / State 시각화 분석

CMT의 attention map과 Vim의 SSM state evolution을 같은 task에서 비교하면, 두 모델이 "어떻게 다르게 보는가"가 직관적으로 드러납니다.

9.1 분류 Task에서의 주의 패턴

CMT의 Stage별 Attention 진화 (Coarse-to-Fine)

CMT의 LMHSA는 stage별로 다른 양상을 보입니다:

Stage	해상도	k (reduction)	Heads	관찰되는 패턴
Stage 1	56×56	k=8	H=1	Texture & edges — locality-dominated (DWConv + reduced K/V)
Stage 2	28×28	k=4	H=2	Body parts — texture clusters
Stage 3	14×14	k=2	H=4	Object regions — semantic grouping
Stage 4	7×7	k=1	H=8	Discriminative parts (예: 새의 머리, 부리) — full attention on 49 tokens

이 패턴은 hybrid 모델의 전형적 행동입니다. **Stage 1~2에서는 DWConv가 지역성을 강제하기 때문에 attention map이 비교적 spread out 되어 있고 텍스처/엣지를 균등하게 봅니다. Stage 3~4에서는 k=2, k=1로 줄어들면서 attention이 almost-full이 되고, 학습된 표현이 충분히 추상화되어 discriminative한 부위에 집중됩니다.**

CMT 논문 자체에는 attention map 시각화가 없지만, 같은 hybrid 계열인 CvT, Swin, PVT 논문들의 시각화 결과를 보면 모두 이 coarse-to-fine 패턴을 보입니다.

Vim의 State Evolution Pattern

Vim에는 attention map이 없습니다. 대신 **hidden state $h_t \in \mathbb{R}^{(N=16)}$ 의 시간 진화**를 시각화합니다.

14×14 패치의 row-major scan을 따라 SSM state norm $\|h_t\|$ 이 변화하는 양상:

- 스캔 시작점에서 0에서 출발
- 객체 영역을 만나기 전: 완만한 누적
- **객체 영역 진입 → state norm이 급격히 상승**
- 객체 영역 통과 후: 완만한 감쇠

핵심 메커니즘: 객체 영역에서 state norm이 급격히 상승하는 것은 selective mechanism이 객체 토큰에서 작은 Δ_t 를 학습해 정보를 stage에 오래 머물게 하기 때문입니다. 객체를 지나가면 다시 감쇠가 빨라져 state norm이 평탄해집니다.

Vim 시각화의 핵심 차이:

- CMT의 attention map은 spatial heatmap (어디를 본다)
- Vim의 state는 temporal signal (시퀀스 진행에 따라 어떻게 누적되는가)

9.2 Detection Task — 큰 객체 vs 작은 객체

Vim 논문 Figure 5는 **큰 객체에서 Vim이 DeiT보다 우수함**을 시각적으로 보였습니다. 비행기 같은 큰 객체에서 DeiT는 일부만 인식하지만 Vim은 전체를 인식합니다. 이 현상의 원인을 두 모델 관점에서 비교:

CMT: 큰 비행기 객체

Stage 4 attention ($k=1$)

- Bias: center of object (중심부 집중)
- Tail and wings underweighted
- 전체 객체 중 discriminative center만 강조

Vim: 같은 비행기

Bidirectional state accumulation

- Bias: scan-trajectory continuous
- Object spans many tokens fluidly
- 전체 body 커버 (whole body covered)

왜 Vim이 큰 객체에서 우위인가?

핵심 통찰 — Softmax의 dilution 효과:

Softmax attention의 본질은 "가중치 합이 1"인 분포입니다. 객체가 K개 토큰에 걸쳐 있으면, 각 토큰의 평균 가중치는 $1/K$ 로 줄어듭니다. 매우 큰 객체에서는 **information dilution** 문제가 발생합니다.

수학적으로:

$$y_i = \sum_j \text{softmax}(q_i k_j) v_j, \quad \sum_j \text{softmax}(q_i k_j) = 1$$

만약 $K=50$ 개 토큰이 동일하게 중요하면, 각자 weight 0.02 — 매우 작은 영향력.

반면 SSM은 cumulative입니다:

$$h_t = \bar{A} h_{t-1} + B x_t$$

객체 토큰을 만날 때마다 state에 additive하게 추가됩니다 (Bx_t 항). 50개 토큰을 거치면 state는 50개의 정보가 누적된 형태. Dilution 없음 — 단지 멀리 떨어진 토큰은 $\bar{A}^k$ 로 감쇠하지만, 양방향이므로 객체 중심에서 양쪽 끝까지의 거리는 $K/2$ 로 줄어듭니다.

이것이 **Vim 논문 Figure 5**에서 비행기 전체를 잡는 메커니즘입니다. Result: +1.4 AP_box_large for Vim-Ti vs DeiT-Ti on COCO.

반대로 작은 객체에서는?

모델	작은 객체 AP_S	큰 객체 AP_L
DeiT-Ti	18.1	58.4
Vim-Ti	18.2 (+0.1)	60.2 (+1.8)
차이	미미	명확한 우위

해석: 작은 객체는 몇 개 토큰만 차지하므로 attention의 dilution 문제가 발생하지 않습니다. 오히려 dense feature가 더 중요한데, 이는 CMT의 multi-scale stage 구조가 유리한 영역입니다. CMT가 RetinaNet 기반에서 mask AP에서 우위를 보인 것(40.7 vs Vim 39.2)도 같은 맥락 — 작은/중간 객체의 정밀한 경계 처리.

9.3 Semantic Segmentation의 픽셀 단위 동작

ADE20K 같은 segmentation에서는 **pixel-level decision**이 필요합니다. 두 모델이 같은 픽셀에 대해 어떻게 판단하는지 비교:

CMT: Hierarchical feature fusion

- Stage 1: 56×56 , fine detail (boundary 정보)
- Stage 2: 28×28 , parts
- Stage 3: 14×14 , objects
- Stage 4: 7×7 , semantics
- FPN fusion → per-pixel logits (upsample + skip)
- Boundary pixels benefit from Stage 1 high-resolution features

Vim: Single-scale + bidir context

- Single resolution: $H/16 \times W/16$
- 24 bidirectional SSM blocks
- Each pixel sees entire image context
- UperNet head: upsample to full resolution
- Boundary pixels: rely on token's own selective Δ , no explicit fine scale

결과 (ADE20K, UperNet, 512^2 input):

- DeiT-S: 44.0 mIoU
- Vim-S: 44.9 mIoU (+0.9)

Vim wins despite lacking native multi-scale features. **Trade-off:** CMT의 stage features가 sharper boundaries를 제공하고, Vim의 bidir context는 better long-range consistency를 제공.

Segmentation에서의 두 모델의 실패 모드:

- **CMT의 전형적 실패:** 큰 동질 영역의 일관성. 예를 들어 "하늘"이 이미지 전반에 걸쳐 있을 때, Stage 1~3의 multi-scale 처리가 부분적으로 다른 라벨을 예측할 수 있음. FPN fusion이 이를 부분적으로 해결하지만 완벽하지 않음.
- **Vim의 전형적 실패:** 경계의 정확성. 단일 H/16 해상도로 작업하므로 미세한 경계는 UperNet upsampling에 의존. 작은 객체나 얇은 구조(전선, 줄기 등)에서 boundary가 흐릿할 수 있음.

이는 후속 연구 **VMamba**가 4-stage hierarchical + 2D selective scan을 도입해 두 약점을 동시에 해결하려 한 동기가 됩니다.

9.4 시각화 도구 및 비교 실험 설계

시각화를 통한 핵심 차이 종합

측면	CMT 시각화 특성	Vim 시각화 특성
무엇을 본다	Attention map (공간 분포)	State trajectory (시간 진화)
시각화 단위	Stage별 attention pattern	시퀀스 위치별 $\ h_t\ $
객체 중심 패턴	Center-biased (discriminative parts)	Trajectory-continuous
큰 객체	Dilution으로 일부만 강조	전체 누적, full coverage
작은 객체	Multi-scale로 정밀	Single-scale 한계
경계 처리	Stage 1 fine feature 활용	Selective Δ 에 의존
시각적 해석 가능성	Heatmap으로 직관적	State norm/PCA로 추상적

실제 시각화를 위한 도구와 방법

CMT용 도구

- **Attention rollout** (Abnar & Zuidema, 2020): 모든 레이어의 attention matrix를 곱해 "입력 토큰이 출력에 얼마나 기여했는가" 계산
- **Grad-CAM** (Selvaraju et al., 2017): 마지막 conv layer의 gradient로 class activation map 생성
- **Attention head visualization:** 각 head가 무엇을 보는지 분리 시각화
- 라이브러리: timm의 hook + pytorch-grad-cam

Vim용 도구

- **Hidden state norm tracking:** $\|h_t\|_2$ 를 시퀀스 따라 plot
- **Δ_t heatmap:** 토큰별 학습된 Δ 값을 2D grid로 visualize → "어디서 천천히 읽는가"
- **Effective attention** (Ali et al., 2024): SSM을 attention 형태로 reformulation. $w(t,s) = C_t \cdot \tilde{A}_{t:s+1} \cdot B_s$ 계산
- **State PCA:** 16차원 state를 PCA로 2D 투영해 trajectory 시각화

- 라이브러리: mamba-ssm 공식 코드 + custom hook

10. 비교 논문 작성을 위한 실험 제안

10.1 권장 통제 실험

비교 논문을 쓰신다면 다음 실험이 강력한 contribution이 될 수 있습니다:

실험 1 — 통제된 정확도 비교

동일 학습 설정(300 epochs, DeiT augmentation, AdamW)에서 두 모델을 동일 파라미터 예산으로 재학습. 이 미 두 논문이 DeiT를 기준으로 삼았으므로 DeiT를 공통 기준으로 정렬 가능.

실험 2 — 해상도 sweep

224^2 , 384^2 , 512^2 , 768^2 , 1024^2 에서 정확도/메모리/속도를 측정. Cross-over point를 실증하고 "어디서부터 Vim이 유리한가"를 정량화.

실험 3 — Inductive bias 격리 실험

- Vim + CNN stem (CMT의 stem만 차용)
- Vim + hierarchical 4-stage 구조
- CMT의 LMHSA를 SSM으로 교체

이를 통해 정확도 차이가 구조에서 오는지 연산자에서 오는지 분리.

실험 4 — 시각화 비교

Attention map (CMT) vs SSM state evolution (Vim) — 동일 이미지에 대해 두 모델이 "어디를 보는지" 시각화. Vim 논문 Figure 5처럼 large object에서 SSM이 우수한지, small object에서는 attention이 우수한지 등.

실험 5 — Robustness/Generalization

ImageNet-C (corruption), ImageNet-A (adversarial), 도메인 시프트 데이터셋에서의 성능. Inductive bias가 강한 CMT가 더 robust할지, sequence modeling이 일반적인 Vim이 더 robust할지는 흥미로운 열린 질문.

10.2 시각화 실험 제안

실험 A — 정성적 비교

ImageNet validation에서 5~10개 대표 이미지(작은 새, 큰 동물, occluded object, multi-object scene)에 대해 두 모델의 시각화를 나란히 배치. "Vim 논문 Figure 5처럼 큰 객체에서 Vim이 우수함을 정량화한 정성 figure"가 핵심 contribution.

실험 B — Quantitative ERF 측정

Luo et al.의 방법론으로 실제 effective receptive field를 측정. 입력에 작은 perturbation을 주고 출력 변화를 측정하여 ERF map 생성. CMT는 Gaussian-like, Vim은 scan-aligned anisotropic 패턴이 나올 것으로 예상.

실험 C — Token importance ranking

ImageNet에서 "가장 영향력 있는 토큰 top-10"을 두 모델별로 추출. CMT는 attention score, Vim은 $\|B_t\| \cdot \|x_t\|$ 로 측정. 두 모델이 같은 토큰을 선택하는 비율(IoU)을 계산하면 "두 모델이 같은 정보를 사용하는가"의 정량 지표.

실험 D — Failure case analysis

ImageNet에서 두 모델이 모두 틀린 케이스, 한쪽만 맞는 케이스를 시각화. 두 모델의 complementarity를 드러내고 ensemble 가능성을 제시.

이 네 실험은 논문의 visualization section 4페이지 분량을 채울 수 있는 contribution이며, 단순한 "수치 비교"를 넘어 "두 모델이 본질적으로 어떻게 다르게 보는가"를 보여주는 강력한 증거가 됩니다.

11. 종합 평가와 후속 연구 맥락

두 논문을 "같은 트랙의 경쟁자"로 보는 것은 부분적으로만 맞습니다. **CMT(2021)**는 ViT-DeiT 시대의 정점으로서 "Transformer에 CNN을 어떻게 잘 섞을 것인가"의 모범 답안을 제시했고, **Vim(2024)**은 Mamba 시대의 시작점으로서 "attention 없이도 가능하다"는 가능성을 제시했습니다.

비교 논문 작성을 위한 핵심 프레임

1. **공통 문제 (Common Problem)**: ViT의 quadratic complexity와 inductive bias 부재
2. **분기점 (Divergence)**: Hybrid(CMT) vs Replacement(Vim) — 이 대비를 메인 thesis로
3. **체계적 비교 축**:
 - 복잡도 (이론적 + 실측 FPS/memory)
 - Inductive bias의 강도와 효과
 - Multi-scale 표현의 필요성에 대한 입장
 - 위치 정보 처리 방식 (CMT: relative bias + DWConv vs Vim: position embedding + 양방향 SSM)
4. **공정성 확보**: 동일 데이터셋·동일 framework로 재실험할 수 있다면 이상적. 어렵다면 두 논문의 보고치를 사용하여 framework 차이를 명시.
5. **미래 방향**: 두 접근의 융합 가능성 — 예: 계층적 SSM 백본, SSM + DWConv hybrid (실제로 VMamba, LocalMamba 등 후속 연구가 이 방향).

후속 연구 흐름

2024년 이후 SSM 기반 비전 모델 연구가 폭발적으로 증가한 맥락을 고려하면, Vim은 "새 패러다임의 출발점"으로, CMT는 "하이브리드 접근의 정점"으로 자리매김하는 서사가 자연스럽습니다.

- **CMT 계열**: Swin → CSwin → MaxViT → EfficientFormer로 이어지며, hybrid 설계가 산업계 표준이 됨.
- **Vim 계열**: VMamba(2024.1, 2D scan), LocalMamba(2024.3, window-based scan), PlainMamba, EfficientVMamba 등 폭발적 증가. 2024년 한 해 동안 SSM 기반 비전 모델만 50개 이상 발표.

특히 **VMamba**는 Vim의 평면 구조 한계를 인식하고 4-stage hierarchical + 2D selective scan을 도입했는데, 이는 사실상 "**Vim에 CMT의 구조적 교훈을 이식한**" 시도입니다. 이는 두 논문의 통찰이 상호 보완적임을 보여줍니다.

최종 핵심 요약

Receptive field 관점:

- CMT는 "hierarchical하게 ERF를 키우는 CNN 전통" + "한 블록 만에 전역 도달하는 attention"의 결합. ERF가 2D-isotropic.

- Vim은 "단일 블록에서 전체 시퀀스 도달"이지만 1D scan 방향에 정합된 anisotropic ERF. 이것이 후속 VMamba/LocalMamba가 multi-direction scan을 도입한 이유.

Discretization 관점:

- CMT: 시간/공간 스케일이 hand-designed (stage stride, LMHSA k값). 안정적이지만 데이터 적응성 부족.
- Vim: ZOH 기반 learnable discretization을 통해 "시간 스케일 자체를 입력으로부터 학습". 더 일반적이지만 학습 비용 증가.

수학적 패러다임 관점:

- CMT는 "convolution + softmax attention의 가산적 결합" — 표현력이 크지만 inductive bias 강제.
- Vim은 "입력 의존적 linear time-variant 동역학 시스템" — 형식적으로 attention의 부분집합이지만, ZOH와 selective mechanism으로 실용적 강력함 확보.

시각화 관점:

- CMT의 attention map은 spatial heatmap — 어디를 보는가
- Vim의 state evolution은 temporal signal — 시퀀스 진행에 따른 누적 양상
- 큰 객체에서는 Vim이 우위 (softmax dilution 회피), 작은 객체와 boundary에서는 CMT가 우위 (multi-scale feature)

비교 논문의 Theoretical Analysis 섹션에서 위 세 관점을 축으로 잡으면, 단순한 "수치 비교 + 논문 요약"을 넘어서는 **원리적 비교**가 가능합니다. 특히 (1) ERF의 형태가 down-stream task에 미치는 영향, (2) Learnable discretization이 데이터 효율성에 미치는 영향, (3) 표현력과 일반성의 trade-off — 이 세 가지를 실험적으로 검증하면 논문 기여도가 크게 올라갈 것입니다.